

JARVIS

Personal Operating System

Product Requirements Document V2

Draft for owner review
Documentation-only product direction
2026-09-04

*Canonical source: repository Markdown
This PDF is a human-readable snapshot.*

Contents

- Part I - Product Requirements Document V2
- Part II - Current Implementation Handoff
- Part III - Open Decisions and Owner Questionnaire
- Part IV - Roadmap V2

Supporting architecture, device, privacy, app, web, executor, ADR, and repository integration documents are included in the accompanying Markdown pack.

Part I

PRD V2 PERSONAL OS

Product Requirements Document V2

Status: Draft for owner review

Owner: Naim

Audience: Product owner, Codex, future implementation agents, security reviewers, and JARVIS maintainers

Relationship to PRD V1: Extends and reframes the long-term product. It does not erase the original V1 requirements or automatically authorize code changes.

1. Executive summary

JARVIS is a private, single-user Personal Operating System. It combines a persistent life model, executive assistance, accountability, planning, memory, historical recall, voice interaction, device context, first-party personal apps, and a permissioned action layer.

WhatsApp is the first conversational channel, not the final product. The web control center is the first owner-visible administration surface. Android becomes the first deeply integrated JARVIS device. Future first-party apps, device agents, and restricted executors attach to the same canonical JARVIS Core.

JARVIS should know the owner's plans, commitments, people, projects, routines, goals, preferences, and historical context. It should communicate naturally, remain present without becoming noisy, challenge weak excuses, accept valid explicit overrides, replan when reality changes, and gradually earn permission to act.

JARVIS must never become an unbounded model with direct access to devices, money, messages, or shell commands. Intelligence produces structured decisions. Deterministic policy, scoped capabilities, owner authority, approvals, audit, and canonical state control every side effect.

2. Product thesis

Most personal assistants fail because they are primarily chat interfaces. They respond well in the moment but lack persistent commitments, reliable memory, real scheduling, contextual planning, permission boundaries, and durable follow-up.

JARVIS succeeds by treating conversation as one interface into a larger operating system.

The product thesis is:

1. Canonical state matters more than conversational recall.
2. Goals and owner-approved values must survive temporary avoidance.
3. Context should shape tactics without rewriting the mission.

4. Useful intervention requires timing, evidence, and permission, not motivational text alone.
5. Personal data becomes more valuable when it is normalized into a private timeline and connected across first-party apps.
6. The system should earn authority progressively and explain why every action was allowed.
7. The same brain should serve WhatsApp, web, Android, voice, and future devices.

3. User and scope

3.1 Primary user

JARVIS is designed for one owner, Naim. V1 and V2 are not multi-user SaaS products.

3.2 Product relationship

JARVIS acts as:

- executive assistant;
- accountability partner;
- planner and replanner;
- personal memory and historical retrieval system;
- journal and life timeline;
- device-aware voice assistant;
- connector and first-party app coordinator;
- restricted executor after authority is explicitly granted.

3.3 Product boundaries

JARVIS is not:

- a therapist or doctor;
- a financial institution;
- an autonomous money manager;
- an unrestricted browser or shell agent;
- a surveillance service for other people;
- a public chatbot;
- a replacement for owner judgment in high-impact decisions.

4. Core product principles

4.1 One brain, many surfaces

WhatsApp, web chat, Android voice, Android UI, future iPhone, desktop, and first-party apps share one identity, one canonical data model, one conversation ledger, one memory system, one permission model, and one audit history.

4.2 Behavior changes intervention, not mission

Missed actions may change timing, message style, minimum viable action, environment preparation, or recovery planning. Repeated avoidance must not automatically weaken or delete constitution-level goals.

4.3 Silence is not completion

No response may transition an item into waiting, follow-up, escalation, replanning, expiry, or review. It must never become successful completion without explicit evidence.

4.4 Context over rigid rules

JARVIS weighs fixed commitments, consequences, current location, travel time, sleep, recovery, available time, dependencies, owner intent, and alternatives. It does not blindly protect one domain at the expense of the entire day.

4.5 Internal autonomy, external restraint

Low-risk internal organization may occur automatically under policy. External communication, purchases, money movement, irreversible actions, sensitive disclosure, and device administration require stronger authority.

4.6 Minimum necessary data

JARVIS should receive only the context necessary for the current decision. Raw sensitive data may remain local or in a dedicated vault while JARVIS receives a bounded summary or reference.

4.7 Visible uncertainty

Known, inferred, stale, conflicting, missing, and unconnected information remain distinct. JARVIS asks one useful question when ambiguity materially changes the decision.

4.8 Owner inspectability

The owner must be able to see and correct the constitution, memory, timeline, hypotheses, permissions, devices, connectors, actions, and audit history.

4.9 Progressive authority

The system starts read-only and advisory. Authority expands through explicit capabilities, scopes, limits, durations, and approval rules.

4.10 Reversibility first

When uncertain, JARVIS should prefer reversible, lower-impact actions over irreversible actions.

5. Product identity and operational modes

JARVIS has one canonical identity and memory. Named modes change delivery and authority, not the underlying person or database.

5.1 JARVIS mode

Default chief-of-staff mode. Balanced explanation, planning, accountability, and long-term continuity.

5.2 FRIDAY mode

Tactical mode for driving, travel, training, deadlines, or high-pressure moments. Short answers, fast replanning, current-context priority, minimal commentary.

5.3 KAREN mode

Mentor mode for learning, skill development, habit formation, and reflection. More explanation, encouragement, questions, and teaching.

5.4 EDITH mode

Restricted executor mode. Browser actions, APIs, bookings, purchases, device controls, SSH, and other high-authority capabilities. EDITH mode requires stronger capability and approval controls. It is never the default.

5.5 Mode invariants

All modes share:

- the same owner identity;
- the same constitution;
- the same canonical memory and timeline;
- the same policy engine;
- the same audit system;
- the same prohibition on hidden authority escalation.

6. Core user experiences

6.1 Natural conversational capture

The owner may speak or type naturally:

- Dentist Tuesday at 2.
- Do not let me forget the Chase payment.
- Keep bothering me until this is finished.
- Move gym after work.
- Bench was 185 for six.
- I am going home for twenty minutes and then to this address.
- Remind me when I need to leave.
- I watched this movie, eight out of ten.

JARVIS converts statements into structured events, commitments, reminders, plan changes, logs, journal entries, or clarification questions.

6.2 Proactive executive presence

JARVIS should notice and surface:

- deadlines and bills;
- unanswered important email;
- schedule conflicts;
- travel and preparation time;
- training and recovery context;
- open loops;
- commitments at risk;
- unusual charges or duplicate subscriptions when finance is connected;
- important relationship promises;
- stale plans and unreviewed hypotheses.

Normal proactive communication should feel present but not spammy. Related information should be grouped. Critical items may bypass the ordinary message budget.

6.3 Accountability exchange

When the owner tries to drop an important commitment, JARVIS should evaluate:

- constitutional relevance;
- consequences;
- time remaining;
- current constraints;
- alternate windows;
- minimum viable versions;
- previous misses;
- hard overrides;
- tomorrow's protected obligations.

It should challenge once when a realistic option remains. It should not shame, moralize, or default to insults. A clear valid hard override ends ordinary negotiation after one consequence explanation.

6.4 Late wake-up and disruption recovery

A late wake-up, unexpected appointment, work overrun, or missed task triggers a fresh plan. Fixed external anchors and high-consequence commitments are protected first. Flexible and optional work moves, shrinks, or drops with explicit tradeoffs.

6.5 Ghosting

When the owner stops responding:

- low-value conversation decreases;
- commitments remain open;
- one useful check-in may occur;
- one commitment-focused follow-up may occur;
- an evening recovery/reset may occur;

- urgent deadlines may escalate under policy.

6.6 Voice assistant

The owner can say a wake phrase such as "Hey JARVIS" and ask:

- When is my assignment due?
- When is soccer practice?
- How long until I need to leave?
- What did I promise my professor?
- Move the gym session to tonight.
- Start navigation after I leave home.

The system should combine speech, calendar, email, location, travel estimates, preparation habits, and current commitments rather than returning a single isolated fact.

6.7 Historical recall

The owner can ask:

- Where was I on May 26, 2027?
- What was I doing during summer 2027?
- What music did I listen to while driving that month?
- Which movies did I love that year?
- When did this relationship conflict start?
- What was I working on before this project stalled?

JARVIS reconstructs answers from evidence in the Life Ledger, Journal, memory, connected apps, and retained source references. It distinguishes exact records from inferred summaries.

6.8 Owner control center

The owner can inspect:

- today and upcoming plans;
- commitments and reminders;
- Brain memory and hypotheses;
- Life Ledger and Journal;
- approvals and actions;
- devices and capabilities;
- connectors and sync health;
- model activity and costs;
- audit and security history.

7. JARVIS Core requirements

7.1 Canonical state

A JARVIS-owned canonical database stores durable state. Conversation providers, model providers, Convex, device caches, and external apps are not canonical truth.

7.2 Event-oriented architecture

Every meaningful observation, user message, connector update, plan change, action proposal, approval, execution, and result becomes a typed event or durable state transition with source and timestamps.

7.3 Brain request lifecycle

The Brain must:

1. persist the inbound event;
2. retrieve owner-scoped context;
3. assemble a bounded context manifest;
4. select versioned prompt modules;
5. choose a model route under budget;
6. receive strict structured output;
7. validate all references and action intents;
8. persist the safe decision and evidence;
9. pass actions through policy and approval;
10. create follow-up jobs and a user-visible response;
11. remain idempotent on replay.

7.4 Prompt architecture

Prompts are versioned modules, not one universal instruction blob. Required modules include:

- core identity;
- constitution;
- memory rules;
- accountability;
- planning and replanning;
- behavioral interventions;
- reminders and ghosting;
- communication style;
- action and tool rules;
- uncertainty;
- security;
- privacy.

7.5 Personality learning

JARVIS may learn bounded delivery preferences:

- message length;
- directness;
- humor;

- profanity tolerance;
- strictness;
- praise style;
- explanation depth;
- choice count;
- reminder density;
- morning/evening style.

One emotional interaction must not create a permanent preference. Traits require repeated evidence or owner review and remain editable, freezeable, resettable, and disableable.

8. Constitution, memory, world state, Life Ledger, and Journal

8.1 Personal constitution

The constitution contains owner-approved goals, rules, boundaries, promises, and standing prohibitions. Each item is versioned and auditable.

The model may propose a constitution candidate but may not activate, weaken, replace, or delete an active constitution item.

8.2 Structured memory

Memory types remain distinct:

- permanent facts;
- preferences;
- people and relationships;
- projects;
- observations;
- hypotheses;
- open loops;
- personality traits.

8.3 Ephemeral world state

Current context is temporary and may include:

- current location or place class;
- travel state;
- foreground app;
- screen lock state;
- battery and charging;
- headphones;
- current calendar block;
- active workout;
- driving state;
- local time and weather context;
- device availability.

World state expires unless promoted to the Life Ledger or another durable record under policy.

8.4 Life Ledger

The Life Ledger records what happened historically without claiming every event is a personal fact or preference.

It supports:

- location episodes;
- travel segments;
- app sessions;
- music listening;
- workouts;
- meals;
- photos and media references;
- movie and show watches;
- calculations;
- conversations and call summaries;
- calendar activity;
- work sessions;
- purchases and bills;
- JARVIS decisions and interventions.

8.5 Journal

Journal entries contain the owner's subjective reflection. JARVIS may suggest prompts based on the day's ledger, but it must not fabricate feelings.

8.6 Promotion rules

Life Ledger events do not automatically become memory. The memory engine may generate a candidate when evidence suggests a stable fact, preference, or pattern. The owner can confirm, reject, or leave it as an observation.

9. Historical reconstruction requirements

9.1 Query levels

Historical answers may use:

1. exact source records;
2. normalized Life Ledger events;
3. episodes and daily summaries;
4. Journal entries;
5. confirmed memory;
6. explicit inference labeled with confidence.

9.2 Evidence and uncertainty

Every historical answer should expose, when useful:

- source coverage;
- confidence;
- missing periods;
- whether the answer is exact or reconstructed;
- relevant source links.

9.3 Cost-aware reconstruction

JARVIS should avoid expensive analysis on every raw record. It should:

- extract metadata first;
- cluster events;
- create episodes;
- analyze representative media;
- summarize daily and monthly periods incrementally;
- run deeper reconstruction only when asked.

10. Voice and Android requirements

10.1 Android companion

The Android app initially provides:

- authenticated text/voice conversation;
- push notifications;
- current-day view;
- location events;
- Health Connect integration;
- device status;
- offline queue;
- encrypted local storage.

10.2 Default assistant phase

A later Android phase may become the selected assistant and support:

- local wake-word detection;
- streaming speech recognition;
- voice activity detection;
- personalized vocabulary correction;
- barge-in and interruption;
- streaming text-to-speech;
- call-style escalation;
- hands-free navigation and planning.

10.3 Device context

Android may expose explicitly granted signals such as:

- app usage duration;
- notification metadata;
- local blocking events;
- location;
- health and fitness records;
- media playback metadata;
- connectivity and battery;
- device policy state.

10.4 On-device processing

Sensitive or high-volume processing should occur locally when practical, including:

- wake word;
- domain/category blocking;
- foreground-app classification;
- basic media metadata;
- local privacy filters;
- raw call/audio handling where legally and explicitly enabled.

11. First-party app ecosystem

JARVIS should expose a first-party SDK and data contracts so personal apps become clients of the same core rather than isolated products.

Candidate apps include:

- JARVIS Journal;
- JARVIS Calendar;
- JARVIS Food;
- Iron & Intervals / training;
- JARVIS Movies;
- JARVIS Calculator;
- Relationship Vault;
- Media and music history;
- research and reading;
- device-control app;
- Android assistant.

All first-party apps share owner identity, canonical event logging, offline sync, encryption, permissions, and audit.

12. Capability and authority requirements

12.1 Capability registry

Every client, device, connector, and executor declares supported capabilities, such as:

- location.read
- health.sleep.read
- notifications.metadata.read
- maps.route.prepare
- apps.social.suspend
- email.read
- email.send
- browser.navigate
- browser.purchase
- payment.charge
- ssh.inspect
- ssh.execute

12.2 Permission layers

Technical permission is not authority. The system separates:

1. platform or OS permission;
2. registered JARVIS capability;
3. owner authorization policy;
4. action-specific approval when required.

12.3 Permission leases

Authority may be:

- one time;
- for a time window;
- for the current activity;
- for a named project;
- under an amount limit;
- limited to a merchant or recipient;
- standing until revoked.

12.4 Authority ledger

Every side effect records why it was allowed:

- actor;
- device/executor;
- capability;
- owner rule or approval;
- scope;
- limits;
- start and expiry;

- result;
- rollback status.

12.5 Reversibility metadata

Every action type declares:

- impact;
- risk;
- reversibility;
- rollback mechanism;
- approval requirement;
- freshness/expiry;
- data sensitivity.

13. Commitment devices and self-control

JARVIS may support graduated intervention levels:

1. remind and explain;
2. increase friction;
3. temporarily block an app/site;
4. require explicit override and reason;
5. notify an approved accountability person;
6. execute a separately enrolled bounded commitment contract.

Autonomous punitive money movement is prohibited by default. A future financial commitment mechanism would require a separate product/security review, fixed caps, grace periods, explicit enrollment, clear cancellation, and no emotional model discretion.

14. Web control center requirements

The owner-only web app should include:

- Today dashboard;
- canonical chat mirror;
- Brain and memory editor;
- constitution editor;
- Life Ledger and Journal;
- projects and commitments;
- plans and calendar;
- approvals;
- capabilities and authority ledger;
- devices;
- connectors;
- activity and audit;
- privacy and retention;

- model/cost controls;
- operational health.

Authentication should be owner-only, preferably passkey/WebAuthn with trusted-device and session management. There is no public registration.

15. Connector and external data requirements

15.1 Email and calendar

Gmail starts read-only. Calendar may create or update approved JARVIS-owned flexible blocks while preserving fixed external events.

15.2 Health

Health data may come from Health Connect, Apple Health/HealthKit, WHOOP, Oura, training apps, or first-party apps. JARVIS does not make unsupported medical claims.

15.3 Finance

Finance remains read-only until a separate authority review. Account data, balances, bills, subscriptions, and duplicate charges may inform reminders. Money movement remains prohibited.

15.4 Media and music

Music and media history should use provider interfaces and local now-playing signals. JARVIS stores normalized listening events and source references, not copyrighted audio.

15.5 Relationship and communications vault

Raw intimate communications may remain in a dedicated encrypted source. JARVIS should receive summaries, commitments, unresolved topics, and source references based on owner policy. Deeper retrieval requires explicit purpose and authority.

15.6 Calls and recordings

Call recording/transcription requires explicit consent controls and jurisdiction-aware implementation. Raw recordings should remain local or in a dedicated encrypted archive by default.

16. Executor and EDITH requirements

A future executor layer may perform:

- browser navigation;
- forms;
- bookings;
- ride requests;
- purchases;
- APIs and MCP tools;

- SSH inspection or commands;
- device administration.

Executor requirements:

- capability allowlist;
- isolated credentials;
- sandboxing;
- dry-run/preview;
- deterministic policy;
- approval and spending limits;
- idempotency;
- evidence and screenshots where appropriate;
- result verification;
- audit;
- rollback where possible;
- emergency kill switch.

The model never receives raw payment credentials or unrestricted shell authority.

17. Privacy, security, and retention

17.1 Data locality classes

Every source or record may be classified as:

- local_only;
- cloud_allowed;
- memory_allowed;
- ephemeral;
- restricted;
- shareable_with_approval.

17.2 Retention tiers

Example policy:

- high-resolution raw sensor data: short or configurable retention;
- compressed timeline episodes: long-term;
- important events and journal entries: long-term;
- raw media: source storage with references;
- sensitive communication archives: dedicated policy;
- derived memories: reviewed and versioned.

17.3 Encryption

Sensitive credentials and high-risk data require encryption at rest and in transit. Connector secrets must never live in ordinary plaintext columns or logs.

17.4 Deletion and export

The owner can export and delete data by source, date range, person, domain, or entire system. Derived records should preserve provenance so deletion can cascade safely.

17.5 No hidden surveillance

JARVIS must clearly indicate active sensors, permissions, local blockers, recordings, and retained history. It may not silently record other people or upload private content merely because an OS permission exists.

18. Nonfunctional requirements

18.1 Reliability

- durable idempotent jobs;
- provider reconciliation;
- no duplicate external effects;
- bounded retries;
- explicit degraded states;
- canonical recovery after device or transport outages.

18.2 Latency

- text acknowledgement should feel immediate;
- voice should stream and allow interruption;
- deterministic work should happen before model calls;
- deep historical reconstruction may take longer and report progress.

18.3 Cost control

- free-tier and hard-spend guards where applicable;
- provider-neutral model routing;
- metadata-first media analysis;
- incremental summaries;
- no silent billable fallback.

18.4 Auditability

Every meaningful decision or side effect records trigger, selected context, model/prompt version, policy result, authority source, action, external result, and follow-up.

18.5 Portability

The canonical data model and first-party SDK should avoid locking core state to WhatsApp, Vercel, Convex, Neon, one phone OS, or one model provider.

19. Success metrics

19.1 Usefulness

- fewer missed deadlines and bills;
- increased completion of protected commitments;
- faster recovery after disrupted days;
- accurate historical recall;
- fewer manual context switches among apps.

19.2 Trust

- low false-completion rate;
- no unauthorized external action;
- clear explanations and audit trails;
- corrections persist;
- owner understands why JARVIS acted.

19.3 Interaction quality

- messages feel timely and personal;
- challenge level matches owner preference;
- voice interaction feels natural;
- low repetition and noise;
- appropriate emotional tone without pretending to be human.

19.4 System quality

- idempotent processing;
- deterministic policy coverage;
- connector recovery;
- secure credential handling;
- bounded cost;
- clean source provenance for historical answers.

20. Rollout strategy

The implementation should expand through reviewed phases:

1. complete current cloud runtime and orchestration;
2. perform first real structured Brain model probes;
3. connect WhatsApp through the reviewed transport path;
4. use JARVIS in real daily life and tune the Brain;
5. build the owner-only web control center;
6. add Gmail and Calendar;
7. add Life Ledger and Journal foundations;

8. define Personal OS Core V2 capabilities, authority, device protocol, and world state;
9. build Android companion and voice;
10. add first-party apps and JARVIS SDK;
11. add restricted EDITH executor capabilities gradually.

21. Explicit non-goals for the current implementation

This PRD does not authorize the current Codex task to:

- redesign the working Brain;
- migrate databases;
- change deployment providers;
- create Android code;
- ingest gallery, relationship, or phone-call data;
- enable purchases or money movement;
- start an executor;
- pair WhatsApp;
- add new runtime secrets.

These documents record direction. Each implementation phase still requires a scoped prompt, ADR review, tests, and security gates.

22. Principal risks

22.1 Overcollection

Mitigation: locality classes, minimum necessary data, source-specific retention, owner visibility, and local processing.

22.2 Incorrect memory or historical reconstruction

Mitigation: typed memory, source provenance, confidence, exact-versus-inferred labels, correction workflows.

22.3 Excessive authority

Mitigation: capabilities, permission leases, authority ledger, risk classes, approvals, spending limits, kill switch.

22.4 Behavioral harm

Mitigation: intervention registry, no shaming default, explicit overrides, bounded commitment devices, no autonomous punitive financial actions.

22.5 Provider lock-in

Mitigation: provider-neutral ports, canonical JARVIS state, first-party SDK, source references.

22.6 Cost growth

Mitigation: tiered analysis, incremental summaries, on-device processing, configurable retention, provider routing and hard budgets.

22.7 Security breach

Mitigation: encryption, least privilege, isolated credentials, local-only sensitive sources, audit, secret scanning, rotation and revocation.

23. V2 acceptance criteria

The long-term V2 vision is satisfied when:

- the same JARVIS identity works through WhatsApp, web, and Android;
- the owner can inspect and correct memory, constitution, timeline, permissions, and actions;
- JARVIS reconstructs historical periods with evidence and uncertainty;
- the Android device agent supplies current context without uncontrolled surveillance;
- first-party apps feed one canonical data fabric;
- authority is scoped, temporary when appropriate, and explainable;
- high-impact actions remain controlled;
- a restricted executor performs approved tasks safely;
- the owner experiences continuity rather than retraining separate assistants.

Part II

CURRENT IMPLEMENTATION HANDOFF

Status date: 2026-09-04

Purpose: Preserve the current build state for a future chat or coding session. This file contains no secret values.

1. Repository

Private repository:

`https://github.com/naimozduman/jarvis.git`

Current permanent GitHub checkpoint:

`cb2284b8a6854cb8ede29ea82aeb1b5846337f9`
feat: prepare JARVIS zero-cost cloud deployment

Preferred local repository:

`C:\Users\localhost\Jarvis-Phase-3-6-LF-Safe-20260831`

Do not use the older OneDrive worktree for active changes.

2. Current local state

The local preferred repository contains intentionally uncommitted Vercel runtime/routing fixes discovered during real deployment.

Important changes include:

- Vercel Fastify deployment endpoint in `apps/api/server.ts`;
- reusable API composition renamed from `src/app.ts` to `src/http-app.ts`;
- obsolete generic `api/[...route].ts` removed;
- Vercel route/prefix handling fixed;
- liveness separated from Neon runtime creation;
- bounded initial database connection handling;
- related tests/docs/bundle validation.

Do not reset, clean, stash, or discard this work without review.

3. Live Neon

Project: `jarvis-staging`
Project ID: `jolly-truth-47196608`
Database: `neondb`
Plan: Free

Migrations through `0006_massive_microchip` were manually applied successfully. The `jarvis` schema and tables are visible.

The runtime role is `jarvis_runtime_staging`. Its password was rotated, and the current pooled connection was entered directly into the Vercel Production `DATABASE_URL` secret. Do not request, print, export, or reuse the migration credential.

4. Live Convex development

Existing development deployment:

HTTP Actions base URL:
<https://determined-retriever-869.convex.site>

The Convex dev backend is healthy and stores opaque orchestration metadata only.

Current missing configuration for the Vercel to Convex direction:

`JARVIS_VERCEL_TO_CONVEX_SECRET`

This secret is absent on both Vercel Production and the Convex development deployment.

5. Live Vercel

Projects:

`jarvis-web`
 - reserved for future web control center

`jarvis-api-staging`
 - Fastify/stateless API
 - Hobby plan

Latest known liveness-green deployment:

Deployment ID: `dpl_6pyZzWr1PkN8cuUJTU43CiPk3MB9`
 Production alias: <https://jarvis-api-staging.vercel.app>
 GET /api/health/live: HTTP 200
 Source: serverless/Lambda

Readiness currently returns:

`configuration` = pass
`database` = pass
`queue` = fail
`model` = not_configured

In the Vercel runtime, the legacy queue label means the Convex orchestration handoff is configured. It is not pg-boss.

Vercel Production currently needs:

`JARVIS_CONVEX_ORCHESTRATION_URL`=<https://determined-retriever-869.convex.site>
`JARVIS_VERCEL_TO_CONVEX_SECRET`=<same new secret stored in Convex dev>

The opposite-direction secret `JARVIS_CONVEX_TO_VERCEL_SECRET` already exists in Vercel and must remain independent.

6. Immediate next operational step

When back at the computer:

1. Generate one strong secret without displaying it.
2. Add it to Convex development as JARVIS_VERCEL_TO_CONVEX_SECRET.
3. Add the same value to Vercel jarvis-api-staging Production as a Secret with the same name.
4. Add JARVIS_CONVEX_ORCHESTRATION_URL to Vercel Production using the nonsecret URL above.
5. Redeploy jarvis-api-staging.
6. Verify liveness remains 200.
7. Verify readiness becomes HTTP 200 with database pass, queue pass, and model not_configured.
8. Stop before callback tests or model inference.

7. AI state

- zero-cost mode is enabled;
- no OPENAI_API_KEY;
- no AI_GATEWAY_API_KEY;
- no manual OIDC token;
- no active model route;
- no live JARVIS model inference yet;
- Vercel public catalog previously identified 15 Free Tier model IDs;
- candidate models remain unproven for structured BrainDecision output.

8. WhatsApp state

- transport architecture exists;
- Evolution remains disabled;
- no QR generated;
- no number paired;
- dedicated JARVIS number is planned;
- Evolution/Baileys security gate must be rechecked before pairing.

9. Brain state

The Brain architecture, prompt modules, memory, constitution, accountability, behavior, planning/replanning, reminders, policy, approvals, audit, and provider-free evaluations are implemented. The next major proof is a real model structured-output test after cloud readiness and callback wiring are green.

10. Documentation integration warning

The PRD V2 pack should be added in a separate docs-only commit after the current runtime work is either safely checkpointed or intentionally paused. Do not mix long-term documentation with unresolved deployment source changes unless reviewed explicitly.

Part III

OPEN DECISIONS AND QUESTIONNAIRE

These questions do not block preserving the product direction. They should be answered before the relevant implementation phase.

A. Life Ledger and historical recall

1. How long should exact location points be retained?
2. Should compressed place visits remain forever?
3. Should location history be cloud stored, local only, or hybrid?
4. Should app-usage history retain app names, categories, or only distraction summaries?
5. What level of gallery access is acceptable?
6. Should selected photos receive permanent descriptions?
7. Should JARVIS create daily summaries automatically or only on request?
8. Should monthly/yearly summaries be owner-confirmed before becoming searchable?
9. What data should be excluded from historical recall by default?
10. Should the owner be able to mark periods as private or sealed?

B. Journal

1. Daily prompt time and frequency?
2. Voice versus text preference?
3. Should JARVIS suggest moods or only ask open questions?
4. Should Journal entries be editable without preserving prior versions?
5. Should relationship Journal entries use a separate vault?
6. Should Journal summaries be model-generated automatically?

C. Voice

1. Preferred wake phrase?
2. Preferred voice and formality?
3. How often may JARVIS escalate into call-style UI?
4. Should voice sessions be stored as transcripts, summaries, or not retained?
5. How should background noise and low-confidence transcripts be handled?
6. Should JARVIS speak longer answers or default to a short answer plus offer details?

D. Android and device context

1. Which data should be local-only?
2. Exact location versus place categories?
3. Notification metadata only or selected contents?
4. Which apps may JARVIS monitor for usage?
5. Which apps may it temporarily block?
6. What override delay is acceptable?
7. Should a dedicated managed phone be considered later?
8. Should device rules continue offline?

E. Accountability and commitment devices

1. Preferred escalation ladder?
2. Which commitments deserve stronger enforcement?
3. Should JARVIS notify an accountability person?
4. Should app/site blocking ever occur automatically?
5. How long should blocks last?
6. Should JARVIS require an override reason?
7. Are bounded financial commitment devices ever acceptable?
8. If yes, what hard cap, grace period, and cancellation rule?
9. Which tone is acceptable during a lapse?
10. When should sympathy override strictness?

F. Relationship Vault

1. Which communication sources may be imported?
2. Raw messages, summaries, or both?
3. Should intimate conversations remain local-only?
4. Can JARVIS identify unresolved conflicts?
5. When may it surface a relationship reminder?
6. Should it ever draft or send a message to a partner?
7. What explicit approval is required?
8. How should the other person's privacy be handled?

G. Calls and recordings

1. Should JARVIS support call transcription?

2. Only calls where all participants consent?
3. Raw audio retention period?
4. Local-only versus encrypted cloud archive?
5. Automatic extraction of commitments?
6. Owner review before summaries enter the ledger?

H. First-party apps

Rank priority:

- Journal;
- Food;
- Training;
- Calendar;
- Movies;
- Calculator;
- Relationship Vault;
- media/music;
- reading/research;
- finance dashboard.

For each app, decide whether JARVIS Core or the app owns the detailed domain record.

I. Web control center

1. Which pages are needed for the first usable release?
2. Passkey-only or backup login method?
3. Should the chat UI be part of the first release?
4. Which Brain records may be edited directly?
5. How much raw audit detail should be visible by default?
6. Should historical map views exist?
7. Should private sources require reauthentication?

J. Capabilities and authority

1. Which actions may be always allowed?
2. Which are allowed once or for a time window?
3. Which require biometric approval?
4. Which are permanently prohibited?
5. What purchase limits are acceptable?
6. Should JARVIS ever message other people autonomously?
7. Which SSH actions are acceptable?

8. How should emergency kill switches work?

K. Modes

1. Do the names JARVIS, FRIDAY, KAREN, and EDITH feel right?
2. Should modes be selected manually, automatically, or both?
3. Which capabilities are exclusive to EDITH?
4. What tone should each mode use?
5. Should the name/voice change while the canonical identity stays the same?

L. Cost and infrastructure

1. At what usage point is paying for a better model acceptable?
2. Preferred monthly model budget once paid usage begins?
3. Should expensive historical analysis ask for confirmation?
4. What data merits paid photo/audio analysis?
5. Which infrastructure services may move to paid tiers first?

Part IV

ROADMAP V2

Status: Product sequence draft.

Guiding rule

Finish the smallest real vertical slice before expanding capability. New power requires stronger authority and privacy controls.

Current milestone: Phase 3.6C cloud wiring

Objectives:

- Vercel Fastify liveness and readiness;
- Neon canonical runtime connection;
- Vercel to Convex orchestration configuration;
- Convex callback tests;
- runtime fixes checkpointed;
- no AI call until non-AI path is green.

Phase 3.6D: first live Brain probe

- configure one verified Vercel Free Tier model;
- prove structured BrainDecision compatibility;
- verify OIDC path;
- verify cost receipt and guard;
- replay same request and prove idempotency;
- stop before WhatsApp.

Phase 3.7: first WhatsApp vertical slice

- reverify Evolution/Baileys security target;
- run local Evolution/Postgres/Redis;
- run local bridge;
- pair dedicated JARVIS number;
- inbound message to Brain and outbound response;
- proactive reminder test;
- offline/reconnect test.

Phase 3.8: real-life Brain tuning

- use WhatsApp daily;
- evaluate tone, strictness, ghosting, memory, and reminders;
- collect owner corrections;

- improve prompt modules and deterministic intervention selection;
- do not expand authority yet.

Phase 4: web control center

- owner-only passkey login;
- Today dashboard;
- chat mirror;
- Brain/memory/constitution editor;
- connectors;
- approvals;
- audit;
- operational health.

Phase 5: Gmail and Calendar

- Gmail read-only;
- Calendar read and controlled write;
- source links and reconciliation;
- email deadlines and follow-ups;
- calendar planning integration.

Phase 5.5: Life Ledger and Journal foundation

- ledger data model;
- event/episode pipeline;
- daily timeline;
- Journal prompts;
- historical query MVP;
- source and retention controls.

Phase 6: Personal OS Core V2 hardening

Before deep device integration, add:

- capability registry;
- authority ledger;
- permission leases;
- reversibility metadata;
- device agent protocol;
- ephemeral world-state model;
- locality/retention enforcement;
- first-party app SDK contracts.

Phase 7: Android companion

- authenticated app;
- chat and push-to-talk;

- notifications;
- location;
- Health Connect;
- offline queue;
- device capabilities;
- no default-assistant status initially.

Phase 7.5: Android voice assistant

- local wake word;
- streaming ASR/TTS;
- personalized vocabulary;
- barge-in;
- navigation and call-style escalation;
- selected assistant role where supported.

Phase 8: first-party apps

Priority candidates:

- Journal;
- Food;
- Iron & Intervals;
- Movies;
- Calendar/planner;
- Calculator;
- media history;
- Relationship Vault.

Phase 9: device policy and self-control

- app usage;
- local VPN/category blocking;
- focus/bedtime rules;
- app suspension where permitted;
- explicit override and escalation policies.

Phase 10: EDITH executor

Start with narrow read or reversible capabilities. Expand to browser, bookings, purchases, device administration, and SSH only after authority controls are proven.

Exit criteria by expansion

A new phase should not begin until the previous one has:

- durable canonical state;
- provider-free tests;

- real smoke test where applicable;
- security review;
- audit and idempotency;
- owner review;
- Git checkpoint and CI.

End of PRD V2 snapshot